KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

uv로 파이썬 시작하기

프로젝트를 만들고, 패키지를 넣고,
다른 컴퓨터에서 그대로 받기

```
$ uv init hello-uv
Initialized project `hello-uv`
$ cd hello-uv
$ uv add fastapi
Installed 11 packages in 5ms
$ uv run main.py
fastapi 버전: 0.162.2
$ uv sync
Resolved 12 packages in 1ms
```

오늘의 순서

1	**왜 필요할까**	받은 프로젝트가 안 돌아가는 이유	3쪽
2	**큰 그림**	python, 패키지, 가상환경, uv	4–7쪽
3	**uv 설치하기**	brew, winget	8쪽
4	**프로젝트 만들고 실행하기**	uv init, uv run, 가상환경 켜기	9–14쪽
5	**패키지 넣고 빼기**	uv add, uv tree, uv remove	15–19쪽
6	**다른 컴퓨터에서 그대로**	GitHub, uv sync	20–23쪽
7	**VS Code에서 Jupyter 쓰기**	확장, ipykernel, 커널 고르기	24–28쪽
8	**예전 방식과 비교**	pip, venv를 uv로 바꿔 치기	29쪽
9	**막힐 때**	자주 만나는 메시지	30–31쪽
10	**실제 상황 예제**	튜토리얼, 노트북, 팀원과 함께	32–34쪽

35쪽은 한 장 요약이에요. 4파트에서 만든 hello-uv 폴더를 끝까지 계속 써요.

1. 왜 필요할까

받은 프로젝트를 실행했더니 오류가 나요

- lee 컴퓨터

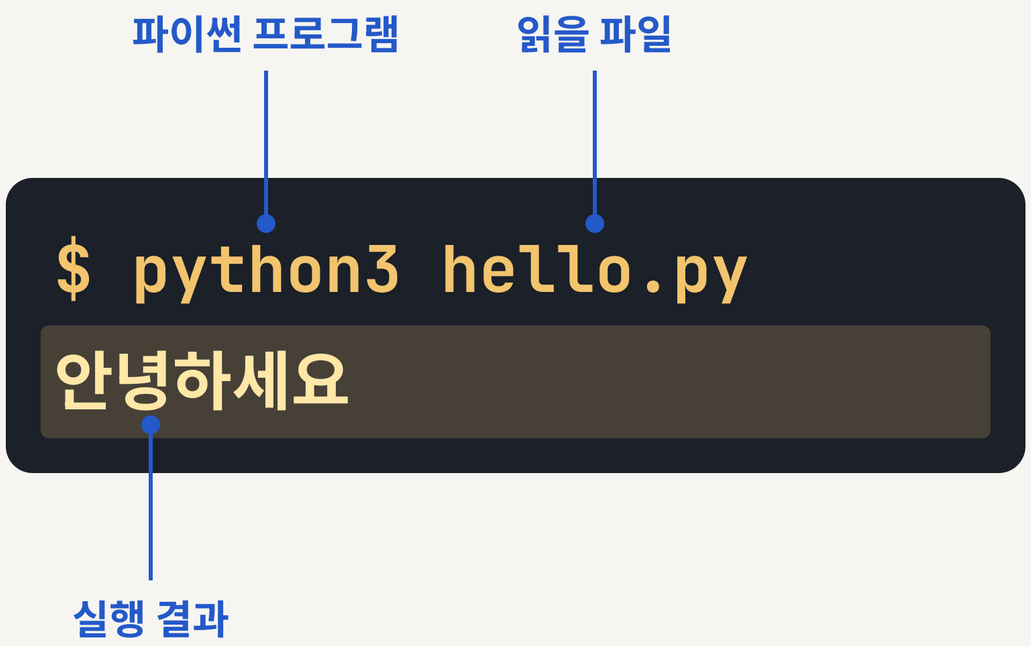
```
$ git clone https://github.com/kim/hello-uv.git
Cloning into 'hello-uv'...
done.
$ cd hello-uv
$ ls -a
.  ..  .git  .gitignore  .python-version  README.md  main.py  pyproject.toml  uv.lock
$ python3 main.py
Traceback (most recent call last):
  File "/home/lee/hello-uv/main.py", line 1, in <module>
    import fastapi
ModuleNotFoundError: No module named 'fastapi'
```

fastapi는 kim 컴퓨터에만 있어요

코드와 목록 파일은 받았는데, 코드가 쓰는 패키지는 따라오지 않았어요.

2. 큰 그림

python이 .py 파일을 읽고 실행해요



hello.py

```
print("안녕하세요")
```

.py 파일은 메모장으로도 열리는 글자 파일이에요.

2. 큰 그림

남이 만든 코드 묶음을 패키지라고 해요

PyPI · 파이썬 패키지 창고

requests	numpy	pandas
openai	**fastapi**	pytest
matplotlib	flask	pillow
scikit-learn	httpx	rich
django	pydantic	polars

… 60만 개 넘게

hello-uv/

main.py

fastapi 받아 온 패키지

fastapi는 웹 서버를 만드는 패키지예요. 이 자료는 fastapi를 넣고, 불러오고, 빼 봐요.

2. 큰 그림

가상환경은 프로젝트마다 따로 쓰는 패키지 상자예요

컴퓨터에 설치한 파이썬 여기에 바로 넣으면 모든 프로젝트가 한 벌을 같이 써요

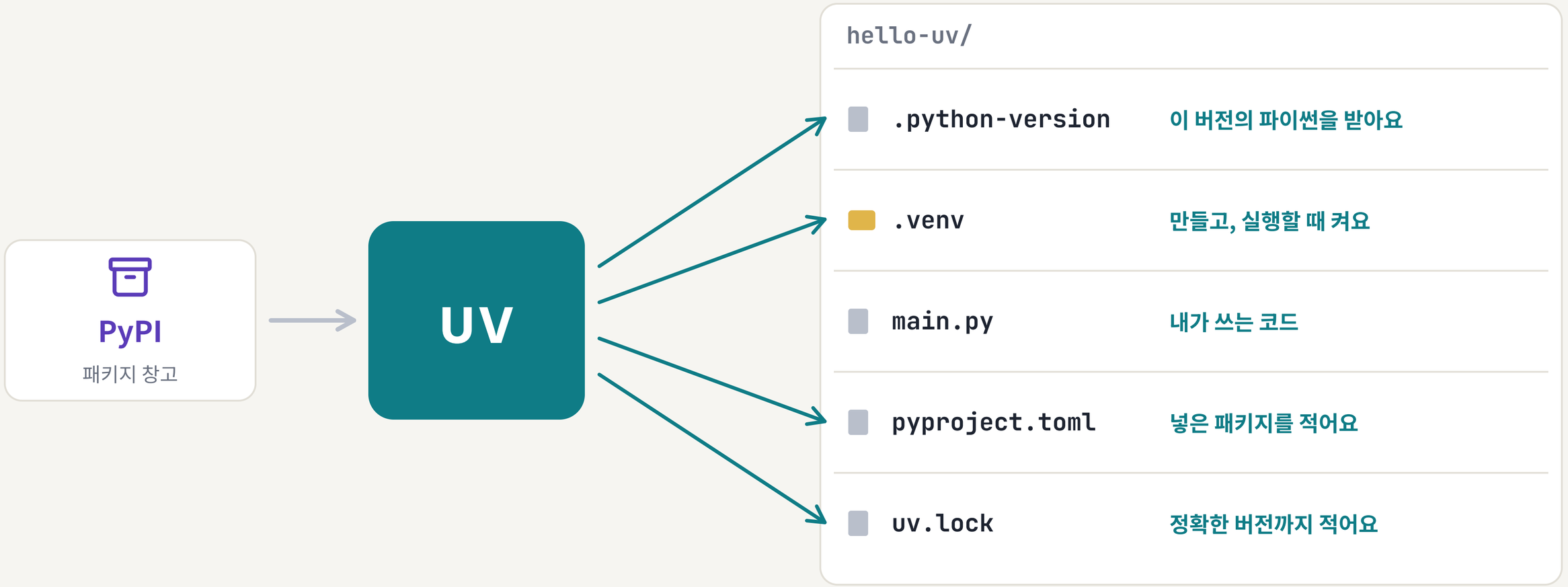
```
project-a/
  main.py
  .venv/
    fastapi 0.162.2
```

```
project-b/
  main.py
  .venv/
    fastapi 0.110.0
```

프로젝트끼리 패키지 버전이 달라도 부딪히지 않아요. uv도 이 .venv를 그대로 써요.

2. 큰 그림

uv가 파이썬, 가상환경, 패키지 목록을 챙겨요



이 목록만 있으면 다른 컴퓨터에서도 같은 .venv를 다시 만들어요.

3. uv 설치하기

명령 한 줄로 uv를 설치해요

컴퓨터	여는 곳	치는 명령
macOS	터미널	`brew install uv`
Windows	PowerShell	`winget install --id=astral-sh.uv -e`

Homebrew가 없으면 brew.sh의 설치 명령을 먼저 쳐요. winget은 Windows 10, 11에 들어 있어요.

! 설치가 끝나면 터미널을 닫았다가 새로 열어요. Windows는 Git Bash를 새로 열어요.

```
$ uv --version
uv 0.11.32 (x86_64-unknown-linux-gnu)
```

4. 프로젝트 만들고 실행하기

uv init으로 프로젝트를 만들어요

```
$ cd ~
$ uv init hello-uv
Downloading cpython-3.14.6-linux-x86_64-gnu
  (download) (34.3MiB) ← 파이썬이 없어서 받았어요
 Downloaded cpython-3.14.6-linux-x86_64-gnu
Initialized project `hello-uv`
  at `/home/kim/hello-uv`
$ cd hello-uv
$ ls -a
```

hello-uv/

.git	Git 저장소 (02)
.gitignore	Git에 안 올릴 목록
.python-version	쓸 파이썬 버전
README.md	프로젝트 소개 글
main.py	실행할 코드
pyproject.toml	프로젝트 설명서

pyproject.toml은 프로젝트 설명서예요

pyproject.toml

```
[project]
name = "hello-uv"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.14"
dependencies = []
```

프로젝트 이름

쓸 수 있는 파이썬 버전

쓰는 패키지, 아직 없어요

.python-version

```
3.14
```

이 폴더에서 쓸 파이썬 버전이에요. uv가 이 버전을 받아서 써요.

4. 프로젝트 만들고 실행하기

uv run으로 실행해요

```
$ uv run main.py
Using CPython 3.14.6
Creating virtual environment at: .venv
Hello from hello-uv!
```

main.py

```
def main():
    print("Hello from hello-uv!")

if __name__ == "__main__":
    main()
```

hello-uv/

.git	Git 저장소 (02)
.gitignore	Git에 안 올릴 목록
.python-version	쓸 파이썬 버전
.venv	가상환경 (패키지 상자)
README.md	프로젝트 소개 글
main.py	실행할 코드
pyproject.toml	프로젝트 설명서
uv.lock	정확한 버전 목록

처음 실행할 때 .venv와 uv.lock이 생겨요. activate는 치지 않아요.

4. 프로젝트 만들고 실행하기

두 번째부터는 바로 실행돼요

```
$ uv run main.py
Hello from hello-uv!
```

1 **켜요**
.venv 안의 파이썬을 골라요

→

2 **실행해요**
main.py를 읽고 실행해요

→

3 **꺼요**
끝나면 원래대로 돌아와요

uv run 한 줄이 이 세 가지를 해요. 프롬프트에 (hello-uv)가 붙지 않아서 끄는 걸 잊을 일이 없어요.

4. 프로젝트 만들고 실행하기

가상환경을 직접 켤 수도 있어요

터미널	켜기
macOS, Linux	`source .venv/bin/activate`
Git Bash	`source .venv/Scripts/activate`
PowerShell	`.venv\Scripts\activate`
끄기	`deactivate`

```
$ source .venv/bin/activate
(hello-uv) $ which python
/home/kim/hello-uv/.venv/bin/python
(hello-uv) $ python main.py
Hello from hello-uv!
(hello-uv) $ deactivate
$
```

켜면 프롬프트 앞에 (hello-uv)가 붙어요.

켜는 걸 잊어도 uv run은 늘 동작해요. 수업 실습은 uv run으로 적어요.

4. 프로젝트 만들고 실행하기

파이썬 버전은 .python-version이 정해요

```
$ cd ~
$ uv init --python 3.13 hello-313
Initialized project `hello-313` at `/home/kim/hello-313`
$ cd hello-313
$ cat .python-version
3.13
$ uv run python --version
Downloading cpython-3.13.14-linux-x86_64-gnu (download) (33.2MiB)
 Downloaded cpython-3.13.14-linux-x86_64-gnu (download)
Using CPython 3.13.14
Creating virtual environment at: .venv
Python 3.13.14
```

버전을 정해서 만들어요

없는 버전은 uv가 받아요

5. 패키지 넣고 빼기

uv add로 패키지를 넣어요

```
$ uv add fastapi
Resolved 12 packages in 1.77s
Downloading pydantic-core (2.2MiB)
 Downloaded pydantic-core
Prepared 11 packages in 437ms
Installed 11 packages in 5ms
 + annotated-doc==0.0.5
 + annotated-types==0.8.0
 + anyio==4.16.0
 + fastapi==0.162.2
 + idna==3.20
 + opentelemetry-api==1.45.2
 + pydantic==2.14.3
 + pydantic-core==2.50.4
 + starlette==1.10.0
 + typing-extensions==4.16.0
 + typing-inspection==0.4.5
```

- **pyproject.toml** 내가 고른 패키지

  "fastapi>=0.162.2",

  dependencies 칸에 적혀요

- **uv.lock** 따라온 것까지 정확한 버전

  고른 1개 + 따라온 10개

  pydantic 2.14.3, starlette 1.10.0 …

- **.venv** 실제로 설치된 패키지

  패키지 11개 설치

  uv run으로 실행하면 바로 써요

5. 패키지 넣고 빼기

하나를 넣었는데 11개가 들어왔어요

```
$ uv tree --depth 2
Resolved 12 packages in 1ms
hello-uv v0.1.0
└── fastapi v0.162.2
    ├── annotated-doc v0.0.5
    ├── opentelemetry-api v1.45.2
    ├── pydantic v2.14.3
    ├── starlette v1.10.0
    ├── typing-extensions v4.16.0
    └── typing-inspection v0.4.5
```

내가 고른 패키지

fastapi가 쓰는 패키지 6개

그 아래 단계에서 4개가 더 들어와서 모두 11개예요.

pyproject.toml에는 내가 고른 fastapi 하나만 적혀요. 뺄 때는 따라온 것까지 함께 빠져요.

5. 패키지 넣고 빼기

넣은 패키지를 코드에서 불러와요

main.py

```
import fastapi


print("fastapi 버전:", fastapi.__version__)
```

.venv에서 fastapi를 찾아 불러와요

```
$ uv run main.py
fastapi 버전: 0.162.2
```

불러온 fastapi의 버전

uv add로 넣은 패키지는 uv run으로 실행하면 바로 불러와요.

5. 패키지 넣고 빼기

uv remove로 패키지를 빼요

```
$ uv remove fastapi
Resolved 1 package in 2ms
Uninstalled 11 packages in 15ms
 - annotated-doc==0.0.5
 - annotated-types==0.8.0
 - anyio==4.16.0
 - fastapi==0.162.2
   … 6줄 생략 …
 - typing-inspection==0.4.5
$ uv run main.py
Traceback (most recent call last):
  …
ModuleNotFoundError: No module named 'fastapi'
```

- **pyproject.toml** 내가 고른 패키지

  dependencies = []

  fastapi 줄이 지워져요

- **uv.lock** 따라온 것까지 정확한 버전

  프로젝트만 남아요

  따라왔던 10개도 지워요

- **.venv** 실제로 설치된 패키지

  패키지 11개 삭제

  딸린 패키지까지 같이 빠져요

5. 패키지 넣고 빼기

다시 넣으면 받지 않고 바로 들어가요

```
$ uv add fastapi
Resolved 12 packages in 8ms
Installed 11 packages in 2ms
 + annotated-doc==0.0.5
 + annotated-types==0.8.0
 + anyio==4.16.0
   … 7줄 생략 …
 + typing-inspection==0.4.5
$ uv run main.py
fastapi 버전: 0.162.2
```

11개가 0.002초 만에

처음 넣을 때 있던 줄

~~Prepared 11 packages in 437ms~~

한 번 받은 패키지는 uv 캐시에 있어서 다시 받지 않아요.

6. 다른 컴퓨터에서 그대로

.venv는 GitHub에 올리지 않아요

```
$ git status --short
?? .gitignore
?? .python-version
?? README.md
?? main.py
?? pyproject.toml
?? uv.lock
$ git add .
$ git commit -m "feat: hello-uv 프로젝트 만들기"
[main (root-commit) a61cccf] feat: hello-uv
  프로젝트 만들기
```



pyproject.toml, uv.lock, .python-version만 있으면 .venv는 똑같이 다시 만들어요.

6. 다른 컴퓨터에서 그대로

lee 컴퓨터에서는 uv sync 한 번이면 돼요

kim 컴퓨터

`pyproject.toml`

`uv.lock`

`.venv`

.venv는 올라가지 않아요

git push origin main

GitHub

`pyproject.toml`

`uv.lock`

git clone 주소

lee 컴퓨터

`pyproject.toml`

`uv.lock`

`.venv`

$ uv sync

파이썬 받기, .venv 만들기, 패키지 11개 설치

uv.lock에 적힌 버전 그대로 설치돼서 kim과 lee의 환경이 같아져요.

6. 다른 컴퓨터에서 그대로

lee 컴퓨터에서 uv sync를 쳐 봐요

- lee 컴퓨터

```
$ git clone https://github.com/kim/hello-uv.git
$ cd hello-uv
$ uv sync
Downloading cpython-3.14.6-linux-x86_64-gnu (download) (34.3MiB)
 Downloaded cpython-3.14.6-linux-x86_64-gnu (download)
Using CPython 3.14.6
Creating virtual environment at: .venv
Resolved 12 packages in 1ms
Prepared 11 packages in 5.63s
Installed 11 packages in 4ms
 + annotated-doc==0.0.5
   … 10줄 생략 …
$ uv run main.py
fastapi 버전: 0.162.2
```

파이썬 받기

.venv 만들기

패키지 11개 설치

6. 다른 컴퓨터에서 그대로

uv sync는 이럴 때 쳐요

	언제	치는 명령	왜

1	clone한 직후	`git clone 주소`
		`cd 폴더`
		`uv sync`
			받은 폴더에는 .venv가 없어요

2	pull로 uv.lock이 바뀐 뒤	`git pull origin main` `uv sync`	목록은 바뀌었지만 내 .venv는 그대로예요
3	잊었다면	`uv run main.py`	uv run이 실행 전에 sync를 알아서 해요

7. VS Code에서 Jupyter 쓰기

노트북은 코드를 셀 단위로 실행해요

notebook.ipynb

[1] 데이터 읽기

표 5줄

[2] 정리하기

빈칸 3개 지움

[3] 그래프 그리기

그래프 그림

셀: 코드를 나눠 담는 칸

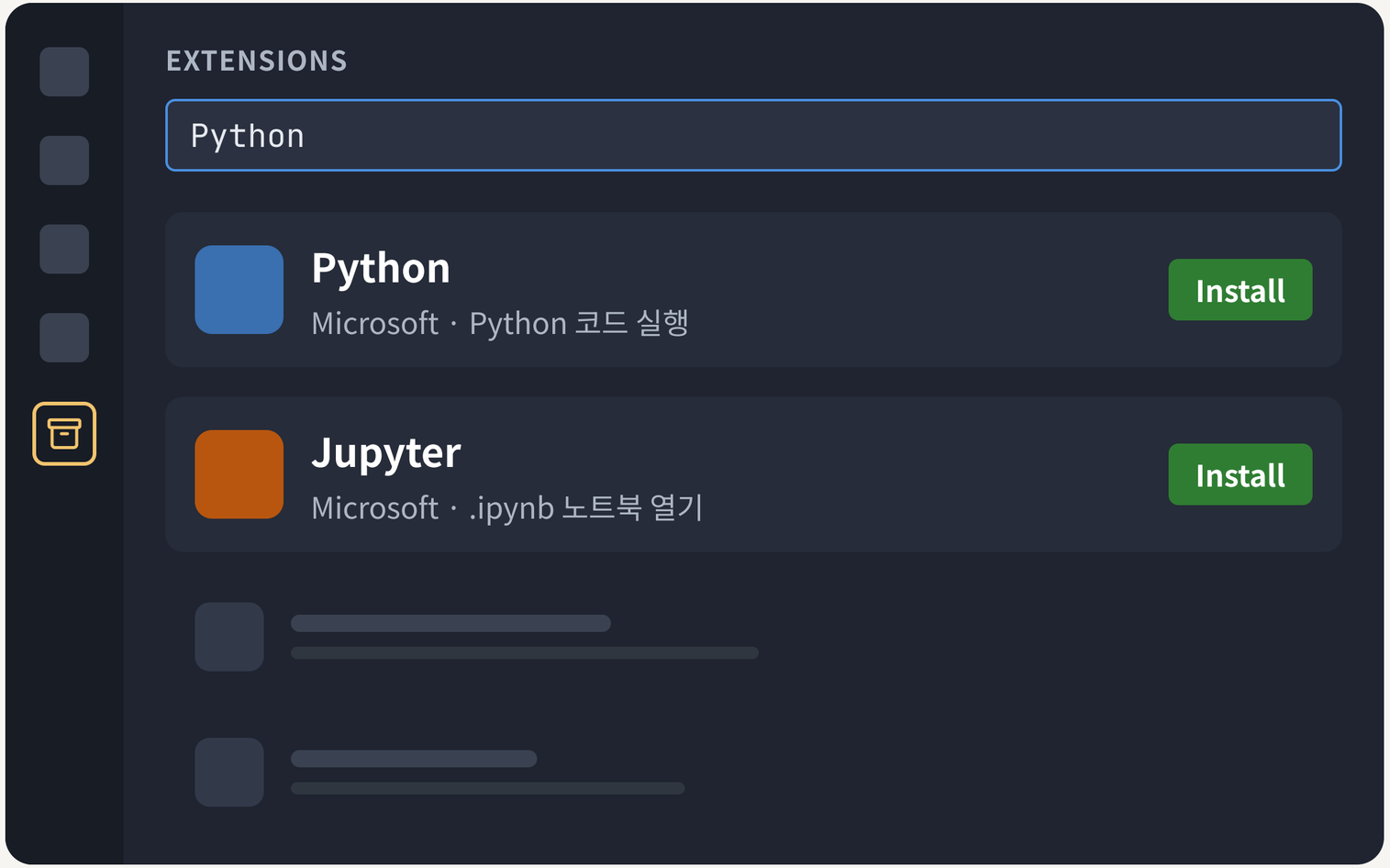
결과가 바로 아래에 남아요

main.py

처음부터 끝까지 한 번에 실행하고, 결과는 터미널에 모여요.

7. VS Code에서 Jupyter 쓰기

VS Code에 확장 두 개를 설치해요



1 **확장 아이콘을 눌러요**

Mac Cmd+Shift+X
Windows Ctrl+Shift+X

2 **Python을 검색해 Install**

만든 곳이 Microsoft인지 봐요

3 **Jupyter를 검색해 Install**

다 되면 VS Code를 다시 열어요

uv add --dev ipykernel로 커널을 넣어요

```
$ cd ~/hello-uv
$ uv add --dev ipykernel
Resolved 44 packages in 1.43s
  … Downloading 줄 생략 …
Prepared 28 packages in 7.29s
Installed 28 packages in 61ms
 + asttokens==3.0.2
 + comm==0.2.3
   … 25줄 생략 …
 + wcwidth==0.9.4
```

pyproject.toml

```
dependencies = [
    "fastapi>=0.162.2",
]

[dependency-groups]
dev = [
    "ipykernel>=7.4.0",
]
```

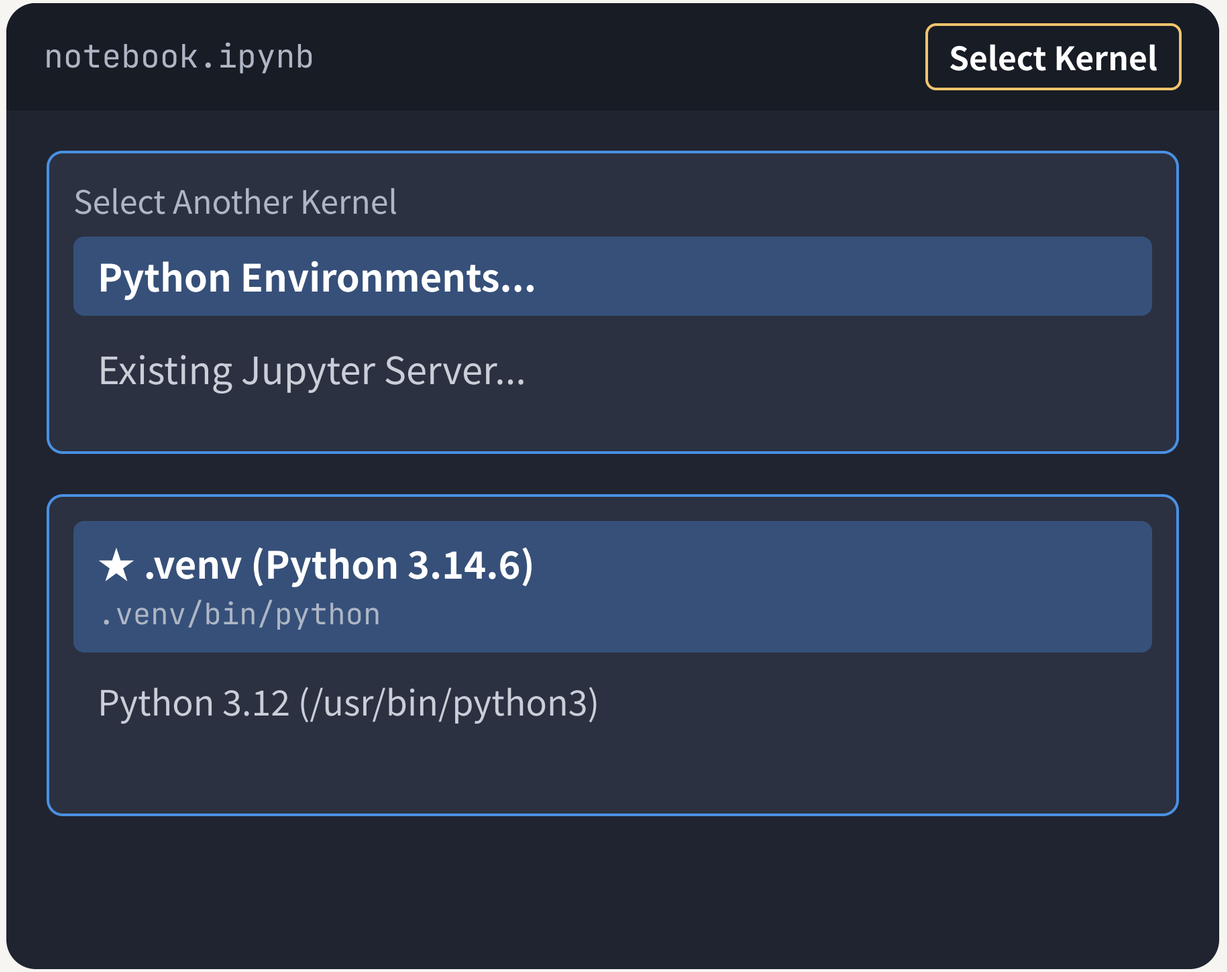
--dev로 넣은 도구는 [dependency-groups]의 dev 칸에 따로 적혀요.

커널은 노트북 셀을 실행하는 파이썬이에요. 노트북도 .venv 안의 파이썬으로 실행하려고 ipykernel을 .venv에 넣어요.

7. VS Code에서 Jupyter 쓰기

노트북을 만들고 커널로 .venv를 골라요

1. **File > Open Folder로 hello-uv 열기**
   폴더 자체를 열어요
2. **탐색기에서 새 파일 notebook.ipynb**
   이름 끝이 .ipynb면 노트북으로 열려요
3. **오른쪽 위 Select Kernel**
4. **Python Environments**
5. **.venv (Python 3.14.6)**
   hello-uv 안의 .venv예요



커널을 한 번 고르면 이 노트북을 열 때마다 기억해요.

7. VS Code에서 Jupyter 쓰기

Shift+Enter로 셀을 실행해요

notebook.ipynb

.venv (Python 3.14.6)

[1]
```
print("안녕하세요")
```

Shift + Enter

```
안녕하세요
```

[2]
```
import sys
print(sys.executable)
```

```
/home/kim/hello-uv/.venv/bin/python
```

.venv면 잘 고른 거예요

[3]
```
import fastapi
print(fastapi.__version__)
```

```
0.162.2
```

[4]
```
1 + 2
```

```
3
```

8. 예전 방식과 비교

인터넷 자료의 예전 명령은 이렇게 바꿔 쳐요

할 일	예전 (따라 치지 않아요)	uv
파이썬 설치	python.org에서 받아 설치	**없으면 uv가 받아요**
가상환경 만들기	`python3 -m venv .venv`	**`uv init 이름`**
가상환경 켜기	`source .venv/bin/activate`	**uv run이 알아서 켜요**
패키지 넣기	`pip install fastapi`	**`uv add fastapi`**
목록 적기	`pip freeze > requirements.txt`	**uv add가 알아서 적어요**
받은 프로젝트 맞추기	`pip install -r requirements.txt`	**`uv sync`**
실행	`python main.py`	**`uv run main.py`**
패키지 빼기	`pip uninstall fastapi` (10개 남아요)	**`uv remove fastapi` (11개 다 빠져요)**

9. 막힐 때

uv에서 자주 만나는 메시지

`bash: uv: command not found`

설치 뒤 터미널을 새로 열지 않았어요

터미널 닫고 다시 열기

`error: No `pyproject.toml` found in current directory or any parent directory`

프로젝트 폴더 밖에서 쳤어요

```
cd hello-uv
```

`ModuleNotFoundError: No module named 'fastapi'`

패키지를 안 넣었거나 python3로 실행했어요

```
uv add fastapi
uv run main.py
```

`error: Project is already initialized in `/home/kim/hello-uv``

이미 만든 프로젝트 안에서 uv init을 쳤어요

uv init은 처음 한 번만

---

9. 막힐 때

노트북과 pull에서 막힐 때

ipykernel을 설치하라는 알림이 떠요

알림의 Install은 pip로 넣어서 pyproject.toml에 안 적혀요.

```
uv add --dev ipykernel
```

커널 목록에 .venv가 없어요

hello-uv 폴더 자체를 열었는지 봐요. 그다음 VS Code를 다시 열어요.

```
uv sync
```

셀에서 ModuleNotFoundError

커널이 .venv가 아니에요. 오른쪽 위 커널 이름을 눌러 바꿔요.

.venv 고르기

`fatal: Need to specify how to reconcile divergent branches.`

pull 설정이 없어요. 한 번만 설정하면 돼요(06).

```
git config --global
  pull.rebase false
```

---

10. 실제 상황 예제

튜토리얼에 pip install이 나오면 uv add로 바꿔요

인터넷 튜토리얼

카페 주문 프로그램을 만들어 봅시다. 먼저 가상환경을 만들고 requests를 설치하세요.

```
~~python3 -m venv .venv~~
~~source .venv/bin/activate~~
pip install requests
python main.py
```

~~uv init과 uv run이 대신해요~~

main.py

```
import requests

print("requests 버전:", requests.__version__)
```

```
$ uv init cafe-order
Initialized project `cafe-order` at `/home/kim/cafe-order`
$ cd cafe-order
$ uv add requests
Using CPython 3.14.6
Creating virtual environment at: .venv
Resolved 6 packages in 223ms
Prepared 4 packages in 309ms
Installed 5 packages in 4ms
 + requests==2.34.2  + urllib3==2.8.0  …
$ uv run main.py
requests 버전: 2.34.2
```

이렇게 넣으면 pyproject.toml에 requests가 적혀서 팀원도 uv sync로 받아요.

10. 실제 상황 예제

노트북에서 없는 패키지는 터미널에서 uv add로 넣어요

notebook.ipynb

[1]
```
import requests
print("requests 버전:", requests.__version__)
```

```
ModuleNotFoundError: No module named 'requests'
```

같은 셀을 다시 실행해요

[2]
```
import requests
print("requests 버전:", requests.__version__)
```

```
requests 버전: 2.34.2
```

- **hello-uv 터미널**

```
$ uv add requests
Resolved 48 packages in 16ms
Installed 4 packages in 3ms
 + certifi==2026.10.23
 + charset-normalizer==3.5.2
 + requests==2.34.2
 + urllib3==2.8.0
```

셀은 노트북에서, 패키지는 터미널에서 넣어요. VS Code 알림의 Install 버튼은 누르지 않아요.

10. 실제 상황 예제

팀원이 넣은 패키지는 pull 뒤 uv sync로 받아요

● kim 컴퓨터

```
$ git status --short
 M pyproject.toml
 M uv.lock
$ git add pyproject.toml uv.lock
$ git commit -m "feat: requests 패키지 넣기"
[main faabfb6] feat: requests 패키지 넣기
$ git push origin main
   0a438ca..faabfb6  main -> main
```

올리는 건 목록 파일 두 개예요. .venv는 올라가지 않아요.

● lee 컴퓨터

```
$ git pull origin main
Fast-forward
 pyproject.toml |   1 +
 uv.lock        | 133 +++++++++…
$ uv sync
Installed 4 packages in 3ms
 + requests==2.34.2  …
$ uv tree --depth 1
hello-uv v0.1.0
├── fastapi v0.162.2
├── requests v2.34.2
└── ipykernel v7.4.0 (group: dev)
```

pull은 목록 파일만 바꿔요. 내 .venv에 실제로 설치하는 건 uv sync예요.

정리

언제 무엇을 치나

이럴 때	이렇게
새 프로젝트를 만들 때	`uv init 이름`
코드를 실행할 때	`uv run main.py`
패키지를 넣을 때 (pip install 대신)	`uv add 패키지`
패키지를 뺄 때	`uv remove 패키지`
받은 프로젝트를 맞출 때 (clone, pull 뒤)	`uv sync`
노트북 커널을 넣을 때	`uv add --dev ipykernel`

GitHub에 올려요 `pyproject.toml` `uv.lock` `.python-version`　　**올리지 않아요** `.venv`

다음 챕터: 10 · VS Code에서 Jupyter 노트북 쓰기. 오늘 만든 hello-uv 노트북을 손에 익혀요.